

1. Contextualização

Esta atividade propõe a refatoração de um sistema de gestão de biblioteca universitária. O código fornecido foi escrito de forma procedural, com diversas violações dos princípios de orientação a objetos, e serve de ponto de partida para a aplicação das regras do Object Calisthenics.

O sistema gerencia empréstimos de livros, multas por atraso, devoluções e geração de relatórios. Cada tarefa foca em um conjunto de regras específicas e solicita a refatoração de um ou mais trechos do código original.

Formato de entrega

O acadêmico deve entregar o projeto Java refatorado, organizado da seguinte forma:

1. Um projeto Maven ou Gradle com estrutura de pacotes coerente com o domínio.
2. Todas as classes refatoradas nas quatro tarefas, em seus respectivos pacotes.
3. O projeto compactado em .zip e enviado via Moodle até a data indicada pelo professor.

Atenção

Preserve a semântica do sistema original — o comportamento deve permanecer equivalente após cada refatoração. O código deve compilar sem erros.

2. Código Original do Sistema

As classes a seguir compõem o sistema que será refatorado ao longo das tarefas. Leia o código integralmente antes de iniciar.

GerenciadorBiblioteca.java

Java

```
public class GerenciadorBiblioteca {

    private List<String[]> livros      = new ArrayList<>();
    private List<String[]> empréstimos = new ArrayList<>();
    private List<String[]> usuarios    = new ArrayList<>();

    // Registra um empréstimo
    public void emprestarLivro(String isbn, String cpfUsr, String dtEmp, String
dtDev) {
        boolean disp = false;
        for (String[] l : livros) {
            if (l[0].equals(isbn)) {
                if (l[2].equals("S")) {
                    disp = true;
                }
            }
        }
    }
}
```

```

        l[2] = "N";
    }
}
if (disp) {
    emprestimos.add(new String[]{isbn, cpfUsr, dtEmp, dtDev, "A"});
} else {
    System.out.println("Livro indisponível ou não encontrado.");
}
}

// Realiza a devolução e calcula multa
public double devolverLivro(String isbn, String cpfUsr, String
dtDevolucaoReal) {
    double multa = 0.0;
    for (String[] e : emprestimos) {
        if (e[0].equals(isbn) && e[1].equals(cpfUsr) && e[4].equals("A")) {
            e[4] = "D";
            int diasAtraso = calcDias(e[3], dtDevolucaoReal);
            if (diasAtraso > 0) {
                if (diasAtraso <= 5) {
                    multa = diasAtraso * 1.50;
                } else {
                    if (diasAtraso <= 15) {
                        multa = diasAtraso * 2.50;
                    } else {
                        multa = diasAtraso * 4.00;
                    }
                }
            }
        }
        for (String[] l : livros) {
            if (l[0].equals(isbn)) { l[2] = "S"; }
        }
    }
    return multa;
}

// Gera relatório de inadimplentes
public void relInad() {
    System.out.println("=== REL. INADIMPLENTES ===");
    for (String[] e : emprestimos) {
        if (e[4].equals("A")) {
            for (String[] u : usuarios) {
                if (u[0].equals(e[1])) {
                    System.out.println("Usr: " + u[1] + " | Livro: " + e[0]
                        + " | Venc: " + e[3]);
                }
            }
        }
    }
}

private int calcDias(String dt1, String dt2) {
    // Implementação omitida – retorna diferença em dias
    return 0;
}
}

```

Principal.java

Java

```

public class Principal {
    public static void main(String[] args) {
        GerenciadorBiblioteca gb = new GerenciadorBiblioteca();
    }
}

```

```

// Adiciona livros: [isbn, titulo, disponivel]
gb.livros.add(new String[]{"978-0-13-468599-1", "Clean Code", "S"});
gb.livros.add(new String[]{"978-0-20-163361-5", "Design Patterns",
"S"});

// Adiciona usuários: [cpf, nome, email]
gb.usuarios.add(new String[]{"12345678900", "Ana Lima",
"ana@puc.br"});
gb.usuarios.add(new String[]{"98765432100", "Bruno
Rios", "brio@puc.br"});

gb.emprestarLivro("978-0-13-468599-1", "12345678900", "2025-03-01", "2025-
03-15");
double multa = gb.devolverLivro("978-0-13-468599-
1", "12345678900", "2025-03-20");
System.out.println("Multa: R$ " + multa);
gb.relInad();
}
}

```

3. Mapeamento de Violações

Antes de iniciar as tarefas, leia o código original e identifique as violações das regras do Object Calisthenics presentes na tabela abaixo. Este mapeamento faz parte do projeto entregue — crie o arquivo `Violacoes.java` ou registre em comentário no topo de cada classe refatorada.

#	Regra	Trecho / método com a violação
1	Um nível de indentação por método	(identificar no projeto)
2	Não usar a palavra-chave <code>else</code>	(identificar no projeto)
3	Encapsular primitivos e Strings	(identificar no projeto)
4	Coleções de primeira classe	(identificar no projeto)
5	Um ponto por linha (Lei de Demeter)	(identificar no projeto)
6	Não usar abreviações	(identificar no projeto)
7	Entidades pequenas (máx. 50 linhas)	(identificar no projeto)
8	Máximo dois atributos de instância	(identificar no projeto)
9	Não usar getters / setters	(identificar no projeto)

4. Tarefas de Refatoração

Cada tarefa delimita o escopo da refatoração e as regras a aplicar. Implemente as soluções diretamente no código Java.

Tarefa
1

Eliminando Primitivos e Abreviações

Regras aplicadas: 3, 6 e 8

10 pontos

Situação

O sistema representa livros, usuários e empréstimos como arrays de String, sem nenhuma validação ou semântica de domínio. Identificadores como isbn, cpf e datas são manipulados como texto puro. Nomes de variáveis e métodos adotam abreviações que prejudicam a leitura.

Trecho de referência

Java — código original

```
// Arrays de String sem tipos semânticos
private List<String[]> livros      = new ArrayList<>();
private List<String[]> emprestimos = new ArrayList<>();
private List<String[]> usuarios    = new ArrayList<>();

// Parâmetros primitivos e método com nome abreviado
public void emprestarLivro(String isbn, String cpfUsr, String dtEmp, String
dtDev) { ... }
public void relInad() { ... }
```

O que implementar no projeto

1. Crie os Value Objects Isbn, Cpf, DataEmprestimo e Titulo, cada um encapsulando o primitivo correspondente com validação mínima no construtor (record ou classe imutável).
2. Crie as entidades Livro, Usuario e Emprestimo utilizando os Value Objects acima. Cada entidade deve ter no máximo dois atributos de instância — use composição para organizar informações adicionais.
3. Renomeie todas as abreviações: relInad() → gerarRelatorioDeInadimplentes(), cpfUsr → cpfUsuario, dtEmp → dataEmprestimo, e assim por diante, mantendo a linguagem do domínio em português.

Tarefa
2

Reduzindo Indentação e Eliminando else

Regras aplicadas: 1 e 2

10 pontos

Situação

O método devolverLivro concentra toda a lógica de devolução em um único bloco com quatro níveis de indentação e dois blocos else encadeados para o cálculo de multa por atraso.

Trecho de referência

Java — código original

```
public double devolverLivro(String isbn, String cpfUsr, String dtDevolucaoReal)
{
    double multa = 0.0;
```

```

for (String[] e : emprestimos) {
    if (e[0].equals(isbn) && e[1].equals(cpfUsr) && e[4].equals("A")) {
        e[4] = "D";
        int diasAtraso = calcDias(e[3], dtDevolucaoReal);
        if (diasAtraso > 0) {
            if (diasAtraso <= 5) {
                multa = diasAtraso * 1.50;
            } else {
                if (diasAtraso <= 15) {
                    multa = diasAtraso * 2.50;
                } else {
                    multa = diasAtraso * 4.00;
                }
            }
        }
        for (String[] l : livros) {
            if (l[0].equals(isbn)) { l[2] = "S"; }
        }
    }
}
return multa;
}

```

O que implementar no projeto

1. Reescreva o método equivalente a `devolverLivro` de modo que contenha no máximo um nível de indentação, extraindo comportamentos para métodos auxiliares com nomes descritivos.
2. Elimine todos os blocos `else` da lógica de cálculo de multa, utilizando cláusulas de guarda (`early return`) ou polimorfismo.
3. Crie a classe ou enum `FaixaDeMulta` que encapsule as faixas de dias e os valores por dia de atraso, eliminando a lógica condicional espalhada.

Dica

Cláusulas de guarda retornam imediatamente quando uma condição não é satisfeita. Em vez de aninhar `if/else`, verifique a condição mais simples primeiro e retorne — o restante do método trata apenas o caso principal.

Situação

Os três campos da classe original são listas de arrays de String com regras de negócio espalhadas pela classe. O método `relInad()` atravessa estruturas internas de dois "objetos" distintos para montar o relatório, violando a Lei de Demeter.

Trecho de referência

Java — código original

```
// Três listas sem encapsulamento:
private List<String[]> livros      = new ArrayList<>();
private List<String[]> emprestimos = new ArrayList<>();
private List<String[]> usuarios   = new ArrayList<>();

// Acessa índices de duas listas distintas — viola a Lei de Demeter:
public void relInad() {
    for (String[] e : emprestimos) {
        if (e[4].equals("A")) {
            for (String[] u : usuarios) {
                if (u[0].equals(e[1])) {
                    System.out.println("Usu: " + u[1] + " | Livro: " + e[0]
                        + " | Venc: " + e[3]);
                }
            }
        }
    }
}
```

O que implementar no projeto

1. Crie `AcervoLivros` como coleção de primeira classe de objetos `Livro`. Exponha apenas comportamentos (ex.: `registrarDisponibilidade()`, `buscarPorIsbn()`, `livrosDisponiveis()`) — sem acesso direto à lista interna.
2. Crie `RegistroDeEmprestimos` como coleção de primeira classe de objetos `Emprestimo`, encapsulando a busca por empréstimos ativos, o registro de devolução e a consulta de inadimplentes.
3. Refatore `gerarRelatorioDeInadimplentes()` para que o método converse apenas com seus vizinhos imediatos — no máximo um ponto por linha de expressão.

Regra da coleção de primeira classe

Uma classe que contém uma coleção não deve ter outros campos além da própria coleção. Se informações adicionais forem necessárias, crie outra classe e componha.

Situação

Com as refatorações anteriores, as entidades Livro, Usuario e Emprestimo foram criadas. É tentador adicionar getters e setters para que a classe principal consiga operar sobre elas — mas isso expõe o estado interno e retorna ao modelo procedural.

Padrão a evitar

Java — padrão a NAO seguir

```
// INCORRETO — getters/setters expõem o estado interno
public class Livro {
    private Isbn isbn;
    private boolean disponivel;

    public Isbn getIsbn()           { return isbn; }
    public boolean isDisponivel()   { return disponivel; }
    public void setDisponivel(boolean d) { this.disponivel = d; }
}

// Código externo que pergunta e age — viola Tell, Don't Ask:
if (livro.isDisponivel()) {
    livro.setDisponivel(false);
    registroDeEmprestimos.adicionar(novoEmprestimo);
}
```

O que implementar no projeto

1. Reescreva Livro sem nenhum getter ou setter. O comportamento de verificar disponibilidade e alterar o estado deve residir dentro do próprio objeto (ex.: `emprestar()` e `devolver()` que lançam exceção se a operação for inválida).
2. Reescreva Emprestimo da mesma forma: o registro de devolução e o cálculo de dias de atraso devem ser encapsulados na classe, não realizados externamente.
3. Refatore o método correspondente a `emprestarLivro` para que ele apenas instrua os objetos a agirem — sem consultar estado e tomar decisões externamente.

Princípio: Tell, Don't Ask

Em vez de perguntar ao objeto sobre seu estado e decidir externamente, instrua o objeto a executar o comportamento. O objeto conhece seu próprio estado e sabe o que fazer com ele.